

Fanuc CRX-10iA Dual-Arm ROS2 System

Technical Report

Ashwin R

January 2026

1 Architecture Overview

This project implements a dual-arm robotic workcell using ROS2 Humble, featuring two Fanuc CRX-10iA collaborative robots with Robotiq 2F-85 grippers.

1.1 System Components

- **ROS2 Humble** — Core middleware with DDS communication
- **ros2_control** — Hardware abstraction and controller management
- **Fake Hardware Interface** — Simulation using `fake_components/GenericSystem`
- **RViz2** — Visualization and debugging

1.2 Controller Architecture

Controller	Type
<code>joint_state_broadcaster</code>	State broadcaster for joint feedback
<code>left_arm_controller</code>	JointTrajectoryController
<code>right_arm_controller</code>	JointTrajectoryController
<code>left_gripper_controller</code>	GripperActionController
<code>right_gripper_controller</code>	GripperActionController

1.3 Package Structure

- `fanuc_crx10ia_description` — Single arm URDF, meshes, and basic launch
- `fanuc_dual_arm_description` — Dual-arm URDF, controllers, Cartesian control, demos

2 Challenges Faced and Solutions

2.1 Docker & Environment Configuration

Challenge: X11 display forwarding failed; RViz/GUI would not render inside the container.

Solution: Enabled host X11 access via `xhost +local:docker` and ensured correct `DISPLAY` environment variable propagation in `docker-compose.yml`.

2.2 Dual-Arm Mounting Orientation

Challenge: Achieving the correct 45° angled mounting for both arms required multiple iterations (pitch vs. roll, sign conventions).

Solution: Used roll rotation (`rpy="0.785 0 0"` and `rpy="-0.785 0 0"`) with Y-axis offset for side-by-side placement, verified visually in RViz.

2.3 Gripper Finger Collision

Challenge: Gripper fingers passed through each other when closing due to incorrect axis directions.

Solution: Corrected the prismatic joint axis and adjusted origin offsets to ensure proper finger travel.

2.4 Cartesian Control Implementation

Challenge: Standard `ros2_control` does not include a built-in Cartesian controller; implementing full inverse kinematics was out of scope.

Solution: Created a custom Python node with simplified geometric IK (sufficient for demonstration), using delta-based motion from a neutral pose to ensure symmetric arm movement.

2.5 Controller Spawning in Launch Files

Challenge: Gripper controllers were not spawning when launching the Cartesian control system.

Solution: Updated `dual_arm_cartesian.launch.py` to explicitly spawn all controllers including grippers.

3 Assumptions and Design Choices

1. **Fake Hardware:** Used `fake_components/GenericSystem` for simulation, allowing development without physical hardware.
2. **Simplified IK:** Implemented geometric inverse kinematics for the demo. Production systems would use KDL, IKFast, or MoveIt.
3. **Symmetric Mounting:** Both arms mounted at ± 45 roll, creating a V-shaped configuration suitable for collaborative tasks.
4. **Delta-Based Motion:** Demo uses position deltas from a neutral pose rather than absolute Cartesian targets, ensuring predictable symmetric motion.
5. **Single Repository:** All packages reside in one repository for simplicity and ease of review.
6. **Docker-First:** Entire system runs in Docker to ensure reproducibility and avoid host dependency conflicts.

References

- ROS2 Humble Documentation: <https://docs.ros.org/en/humble/>
- `ros2_control`: <https://control.ros.org/humble/>
- Fanuc CRX-10iA Specifications: <https://www.fanuc.eu/uk/en/robots/robot-filter-page/collaborative-robots/crx-10ia>
- F. Caccavale et al., "Dual-Arm Jacobian Comparison," IEEE CIS-RAM